

CloudPosture

Cloud Security Posture Management Scanner

Automated AWS Misconfiguration Detection & CIS Benchmark Compliance

Aakash Gadekar

Cloud Security Engineer | Cybersecurity Enthusiast

June 2026

What is CloudPosture?

CloudPosture is a Cloud Security Posture Management (CSPM) tool that automatically scans AWS environments for security misconfigurations. It evaluates your infrastructure against the CIS AWS Benchmark v2.0 - the industry standard for cloud security - and provides a compliance dashboard with actionable remediation guidance.

Think of it as your automated cloud security auditor that checks 50+ security controls across IAM, S3, EC2, RDS, and CloudTrail services, identifies dangerous misconfigurations like public S3 buckets, open security groups, and missing encryption, then tells you exactly how to fix each issue with AWS CLI commands.

The Real-World Problem

Organizations migrating to AWS often leave critical misconfigurations:

- S3 buckets exposed to the entire internet (Capital One breach, 2019)
- Security groups allowing 0.0.0.0/0 on database ports
- IAM policies granting wildcard (*) permissions to developers
- Root accounts without MFA protection
- Databases publicly accessible without encryption
- CloudTrail logging disabled (attackers' first move)

According to IBM X-Force (2025), 80% of cloud breaches are caused by misconfigurations, not sophisticated zero-day attacks. Manual auditing across 200+ AWS services is impossible at enterprise scale. CSPM tools automate this continuous compliance monitoring.

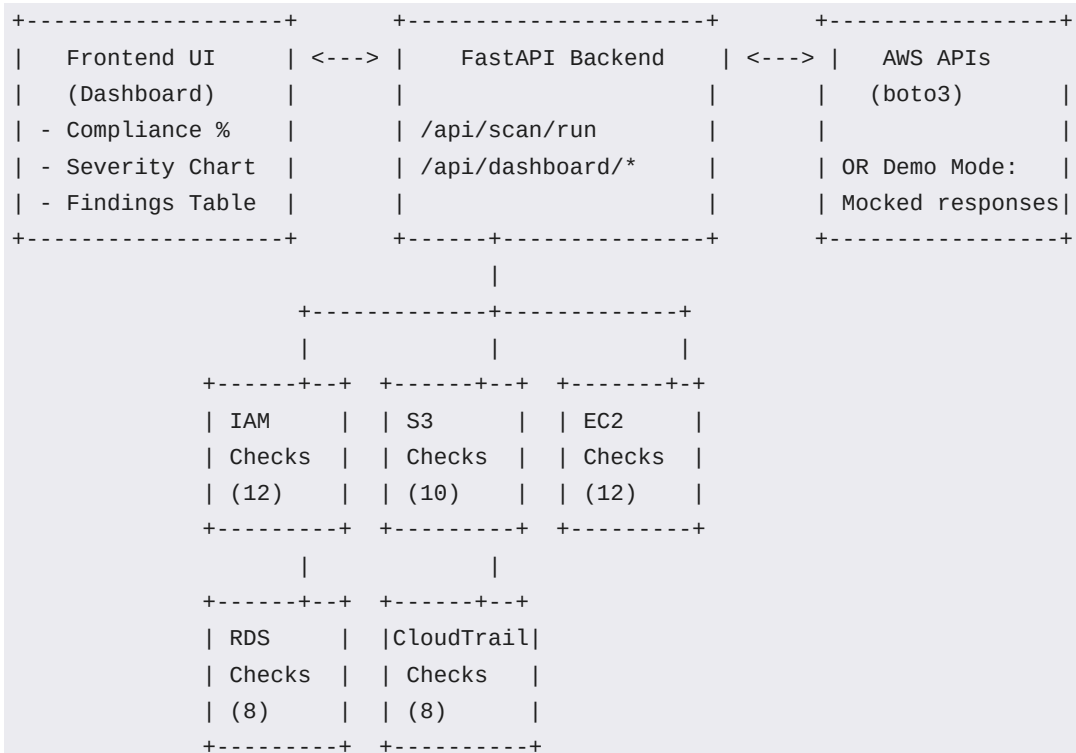
How It Works

1. Scanner Engine triggers parallel security checks across all AWS service categories
2. Each check evaluates a specific CIS Benchmark control (e.g., 'Is root MFA enabled?')
3. Findings are classified as PASS, FAIL, or WARNING with severity ratings
4. Results are compiled into a compliance score (% of checks passing)
5. Dashboard visualizes the posture with charts, tables, and drill-down detail
6. Each failed check includes specific remediation steps and AWS CLI commands

Technology Stack

Technology	Role	Why This Choice
Python 3.11+	Core Language	Industry standard for security tooling
FastAPI	Web Framework	Async support, auto-generated API docs, fast
boto3 (AWS SDK)	AWS Integration	Official SDK for AWS service scanning
Pydantic	Data Validation	Type-safe models for findings/reports
asyncio.gather()	Parallelism	Run 50+ checks simultaneously for speed
Plotly.js	Visualizations	Interactive compliance charts
HTML/CSS/JS	Frontend	Dark-themed SOC-style dashboard
Docker	Deployment	Containerized, one-command deployment
pytest + httpx	Testing	Async API testing

Architecture Diagram



Project Structure

```

CloudPosture/
|-- backend/
|   |-- app/
|   |   |-- main.py          # FastAPI app entry point
|   |   |-- config.py        # Settings (DEMO_MODE toggle)
|   |   |-- models/
|   |       |-- schemas.py    # Finding, ScanResult, Severity enums
|   |       |-- routes/
|   |       |-- scanner.py     # POST /api/scan/run, GET /latest

```

```
| | | |-- dashboard.py      # GET /summary, /findings, /compliance
| | |-- services/
| | | |-- scanner_engine.py  # Orchestrates parallel checks
| | | |-- demo_data.py      # Mocked AWS environment data
| | | |-- checks/
| | | | |-- iam_checks.py    # 12 IAM security checks
| | | | |-- s3_checks.py     # 10 S3 bucket checks
| | | | |-- ec2_checks.py    # 12 EC2/VPC checks
| | | | |-- rds_checks.py    # 8 RDS database checks
| | | | |-- cloudtrail_checks.py # 8 logging checks
| |-- tests/
| | |-- test_scanner.py     # Automated test suite
|-- frontend/
| |-- index.html           # Dashboard page
| |-- static/css/style.css  # Dark theme styles
| |-- static/js/dashboard.js # Charts & interactivity
|-- requirements.txt
|-- Dockerfile
|-- docker-compose.yml
```

What It Covers

Security Check Categories (50+ Total)

IAM Security (12 Checks)

- Root account MFA enabled (CIS 1.5)
- No root account access keys (CIS 1.4)
- Strong password policy - 14 chars, symbols, rotation (CIS 1.8)
- All console users have MFA enabled (CIS 1.10)
- No unused credentials older than 90 days (CIS 1.12)
- No overpermissive policies with Action:* Resource:* (CIS 1.16)
- Access key rotation within 90 days (CIS 1.14)

S3 Storage Security (10 Checks)

- No public bucket access - Block Public Access enabled (CIS 2.1.5)
- Default encryption enabled - SSE-S3 or SSE-KMS (CIS 2.1.1)
- Versioning enabled for data protection (CIS 2.1.3)
- Access logging enabled for audit trail (CIS 2.1.2)

EC2 & VPC Security (12 Checks)

- No security groups with 0.0.0.0/0 on ports 22, 3389, 3306, 5432 (CIS 5.2)
- Default VPC not used for production (CIS 5.4)
- EBS volumes encrypted at rest (CIS 2.2.1)
- IMDSv2 enforced - prevents SSRF credential theft (CIS 5.6)
- VPC Flow Logs enabled for network forensics (CIS 3.7)
- Instances reviewed for unnecessary public IPs (CIS 5.1)

RDS Database Security (8 Checks)

- No publicly accessible RDS instances (CIS 2.3.2)
- Encryption at rest enabled (CIS 2.3.1)
- Multi-AZ for high availability
- Backup retention >= 7 days
- Deletion protection enabled

CloudTrail Logging (8 Checks)

- Multi-region trail enabled (CIS 3.1)
- Log file validation for tamper detection (CIS 3.2)
- CloudWatch Logs integration for alerting (CIS 3.4)
- KMS encryption on log files (CIS 3.5)
- S3 access logging on CloudTrail bucket (CIS 3.6)

Key Learnings & Skills Gained

Cloud Security Knowledge

- > CIS AWS Benchmark v2.0 - The gold standard for AWS security compliance
- > Shared Responsibility Model - What AWS secures vs. what you secure
- > Defense in Depth - Layered security across IAM, network, data, logging
- > Least Privilege Principle - Why wildcard IAM policies are dangerous
- > Encryption at Rest & Transit - S3 SSE, EBS encryption, RDS encryption
- > Network Security - Security groups, VPC design, flow logs
- > Audit & Monitoring - CloudTrail, log integrity, CloudWatch alerting

Engineering Skills

- > Async Python - `asyncio.gather()` for parallel execution of 50+ checks
- > FastAPI - Modern async web framework with auto-generated OpenAPI docs
- > Pydantic Models - Type-safe data validation for security findings
- > Risk Scoring Algorithms - Severity weighting and compliance calculation
- > Full-Stack Development - Backend APIs + interactive frontend dashboard
- > Docker Containerization - Production-ready deployment
- > Test-Driven Development - `pytest` with async HTTP testing

Professional Skills

- > Compliance Reporting - Translating technical findings into compliance scores
- > Remediation Guidance - Writing actionable fix instructions
- > Security Automation - Replacing manual audits with continuous scanning
- > Risk Communication - Severity ratings that prioritize what to fix first

Setup & Installation

Prerequisites

- Python 3.9 or higher
- pip package manager
- (Optional) Docker for containerized deployment

Quick Start (Demo Mode)

```
# Clone the project
git clone https://github.com/aakash1998gadekar/CloudPosture.git
cd CloudPosture

# Create virtual environment
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Run the application
uvicorn backend.app.main:app --reload --port 8000

# Open in browser:
#   Dashboard:  http://localhost:8000
#   API Docs:   http://localhost:8000/docs
```

Docker Deployment

```
docker-compose up --build
# Open http://localhost:8000
```

API Endpoints

POST /api/scan/run	- Trigger new security scan
GET /api/scan/latest	- Get latest scan results
GET /api/scan/status	- Check scan status
GET /api/dashboard/summary	- KPIs, severity breakdown, recent findings
GET /api/dashboard/findings	- All findings with ?service= &severity= filters
GET /api/dashboard/compliance	- Per-service compliance scores

Live Mode (With Real AWS Account)

```
# Set environment variables in .env file:
DEMO_MODE=false
AWS_ACCESS_KEY_ID=your-access-key
```

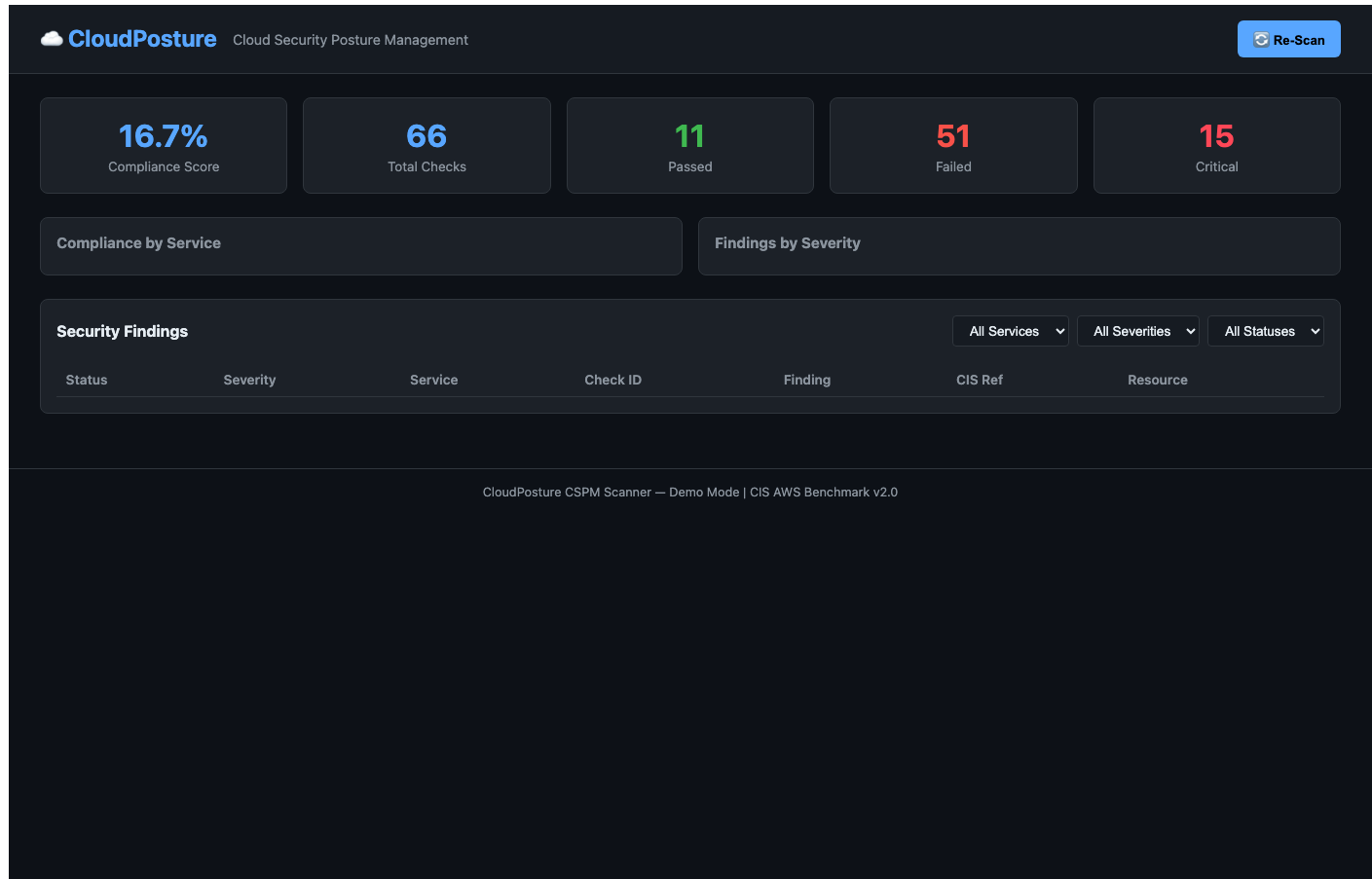
```
AWS_SECRET_ACCESS_KEY=your-secret-key
AWS_REGION=us-east-1

# Requires read-only IAM permissions:
# - iam:List*, iam:Get*
# - s3:List*, s3:GetBucket*
# - ec2:Describe*
# - rds:Describe*
# - cloudtrail:Describe*, cloudtrail:GetTrail*
```

Screenshots

Dashboard View

The main dashboard shows the overall compliance score (16.7% in this intentionally misconfigured demo), KPI cards for total checks/passed/failed/critical counts, a stacked bar chart of findings per service, a severity donut chart, and a detailed findings table with filtering capabilities.



CloudPosture Dashboard - Compliance score, service breakdown, severity distribution, findings table

API Documentation (Swagger UI)

FastAPI automatically generates interactive API documentation. Security teams can integrate CloudPosture into CI/CD pipelines using these REST endpoints to trigger scans and retrieve results programmatically.

CloudPosture — CSPM Scanner 1.0.0 OAS 3.1

[/openapi.json](#)

Cloud Security Posture Management tool for AWS misconfiguration detection

Scanner

POST	/api/scan/run	Trigger Scan	
GET	/api/scan/latest	Get Latest Scan	
GET	/api/scan/status	Scan Status	

Dashboard

GET	/api/dashboard/summary	Get Dashboard Summary	
GET	/api/dashboard/findings	Get All Findings	
GET	/api/dashboard/compliance	Get Compliance Breakdown	

default

GET	/health	Health Check	
-----	---------	--------------	---

Schemas

HTTPValidationError > Expand all **object**

ScanRequest > Expand all **object**

ValidationError > Expand all **object**

CloudPosture Swagger UI - Interactive API documentation with try-it-out functionality

Sample Scan Output

Example findings from a scan of the demo (intentionally misconfigured) environment:

Scan Results Summary:

Total Checks: 66
Passed: 11 (16.7%)
Failed: 51
Warnings: 4

CRITICAL Findings (highest priority):

[IAM-001] Root Account MFA Not Enabled (CIS 1.5)

Resource: arn:aws:iam::123456789012:root

Fix: Enable MFA via IAM console > Security credentials

[IAM-002] Root Account Has Access Keys (CIS 1.4)

Resource: arn:aws:iam::123456789012:root

Fix: aws iam delete-access-key --access-key-id <key-id>

[S3-001] Bucket 'company-public-assets' Has Public Access (CIS 2.1.5)

Fix: aws s3api put-public-access-block --bucket company-public-assets
--public-access-block-configuration BlockPublicAcls=true,...

[EC2-001] SG 'database-sg' Allows 3306 from 0.0.0.0/0 (CIS 5.2)

Fix: aws ec2 revoke-security-group-ingress --group-id sg-0def456ghi
--protocol tcp --port 3306 --cidr 0.0.0.0/0

[RDS-001] 'prod-mysql-primary' Is Publicly Accessible (CIS 2.3.2)

Fix: aws rds modify-db-instance --db-instance-identifier prod-mysql-primary
--no-publicly-accessible

[CT-001] CloudTrail Not Enabled in All Regions (CIS 3.1)

Fix: aws cloudtrail update-trail --name management-trail --is-multi-region-trail